Università degli Studi di Firenze

Dipartimento di Ingegneria dell'Informazione

Applicativo Java EscapeManager

Autore:
Adriano Luca Castaldo

N° Matricola:
7138545

Corso principale:
Ingegneria del software

Docente corso:
Enrico Vicario

GitHub Repository

Anno Accademico 2025/2026

Indice

1 Introduzione **1**
1.1 Statement . . . 1
1.2 Stack Tecnologico e Strumenti . . . 1
1.3 Note metodologiche e Uso dell'Intelligenza Artificiale . . . 2
1.4 Struttura del documento . . . 2
1.5 Limiti del prototipo e sviluppi futuri . . . 2

2 Analisi dei Requisiti **4**
2.1 Use Case Diagram . . . 4
2.1.1 Legenda riferimenti a Mockup e Navigazione . . . 4
2.2 Use Case Templates . . . 5
2.2.1 UC1 - Prenotazione Stanza . . . 6
2.2.2 UC2 - Iscrizione Lista d'Attesa . . . 7
2.2.3 UC3 - Aggiornamento Stato Stanza . . . 8
2.2.4 UC4 - Configurazione Tariffe Stanza . . . 9
2.3 Requisiti non funzionali . . . 9
2.4 Diagramma di Navigazione (CLI) . . . 10
2.5 Mockup e Interfaccia Utente (UI) . . . 10

3 Progettazione Architetturale e di Dominio **13**
3.1 Architettura a Strati e BCE . . . 13
3.2 Package Diagram e Dipendenze . . . 14
3.3 Scelte Progettuali e Trade-off . . . 15
3.4 Domain Model (Class Diagram) . . . 15
3.5 Class Diagram dei Controller e Boundary . . . 16
3.5.1 Scelte Architetturali e Design Pattern . . . 17
3.5.2 State Machine Diagram . . . 17
3.6 Progettazione del Database . . . 18
3.6.1 Diagramma Entity-Relationship . . . 18
3.6.2 Strategie di Mapping e Modello Logico . . . 19
3.7 Dinamica del Sistema (Implementation Perspective) . . . 20
3.7.1 UC1 - Prenotazione Stanza . . . 20
3.7.2 UC4 - Configurazione Tariffe . . . 21
3.8 Pattern Creazionali: DAO Factory . . . 22

4 Implementazione e Architettura del Codice **23**
4.1 Architettura dei Package . . . 23
4.1.1 cli (Boundary) . . . 23
4.1.2 controllers (Control) . . . 23
4.1.3 domain (Entity) . . . 23
4.1.4 dao (Persistence) . . . 23
4.2 Livello di Persistenza (Pattern DAO) . . . 23
4.3 Configurazione della Connessione al Database . . . 24
4.4 Validazione Input e Gestione Errori . . . 24
4.5 Implementazione dei Design Pattern . . . 24

4.5.1 State Pattern 25
4.5.2 Strategy Pattern 25
4.5.3 Observer Pattern 26
4.5.4 Builder Pattern 26
4.6 Dependency Injection e Testabilità 26
4.7 Gestione Centralizzata delle Eccezioni (Information Hiding) 27

5 Collaudo e Testing **29**
5.1 Strategia di Verifica 29
5.2 Setup e Dati di Test 29
5.3 Testing Strutturale (White-Box) 29
5.4 Testing con Mock (Mockito) 30
5.5 Testing di Integrazione (DAO e Database) 31
5.6 Testing Funzionale (Black-Box e Tracciabilità) 31
5.6.1 Catalogo dei Test 31
5.6.2 Tracciabilità Use Case → Test 33
5.7 Risultati 33
5.8 Conclusioni 33

6 Qualità del Software **34**
6.1 Aderenza ai Principi SOLID 34
6.2 Analisi ISO/IEC 25010 34
6.3 Code Coverage (JaCoCo) 35
6.4 Affidabilità e Coerenza dei Dati 35
6.5 Manutenibilità ed Estendibilità 36
6.6 Sicurezza e Privacy (Baseline) 36
6.7 Limiti del Prototipo 36

7 Conclusioni e Sviluppi Futuri **37**
7.1 Obiettivi raggiunti 37
7.2 Sviluppi Futuri 37

1. Introduzione

1.1 Statement

Il progetto **EscapeManager** modella un sistema software centralizzato per la gestione operativa e commerciale di una catena in *franchising* di Escape Room. L'obiettivo principale dell'applicativo è fornire un'infrastruttura unificata che permetta ai clienti di esplorare le stanze tematiche disponibili ed effettuare prenotazioni e, allo stesso tempo, metta a disposizione del personale strumenti per il controllo delle sessioni di gioco e la gestione commerciale.

Le sedi della catena ospitano diverse stanze fisiche (Room), ciascuna caratterizzata da un tema specifico, un livello di difficoltà, una capienza massima di giocatori e un ciclo di vita operativo ben definito (es. *Disponibile, In Corso, In Manutenzione, In Pulizia*).

Il sistema è progettato per supportare tre tipologie principali di utenti:

- **Cliente:** può visualizzare il catalogo globale delle stanze, prenotare una sessione di gioco per la propria squadra in uno slot temporale libero, e iscriversi a una lista d'attesa per eventuali notifiche future in caso di liberazione di uno slot.

- **Game Master (Operatore):** gestisce il flusso operativo quotidiano. Può monitorare le prenotazioni del giorno, avviare o terminare le sessioni di gioco, aggiornare dinamicamente lo stato delle stanze (es. bloccando una stanza per un guasto tecnico a un enigma) e registrare l'esito della partita.

- **Admin (Amministratore):** possiede una visione d'insieme sull'intera catena. Ha i privilegi per configurare politiche di prezzo dinamiche (es. tariffe base o maggiorate).

Per garantire manutenibilità e scalabilità, l'architettura segue il pattern BCE (Boundary-Control-Entity, approfondito nel Capitolo 3) e fa uso di principi di Object-Oriented Design e di Design Pattern strutturali e comportamentali per risolvere problemi specifici del dominio, come la gestione delle variazioni di stato delle stanze, l'applicazione delle tariffe e il sistema di notifiche. La persistenza dei dati, fondamentale per evitare sovrapposizioni nelle prenotazioni e mantenere lo storico delle partite, è gestita tramite un database relazionale connesso al sistema.

1.2 Stack Tecnologico e Strumenti

Per la realizzazione del sistema sono state adottate tecnologie standard in ambito enterprise, al fine di garantire robustezza e manutenibilità:

- **Linguaggio di Programmazione:** Java (JDK 25), scelto per il forte supporto al paradigma Object-Oriented e l'ampia disponibilità di librerie.

- **Persistenza (RDBMS):** PostgreSQL. L'interazione con il database avviene tramite connettore JDBC, con un layer DAO (Data Access Object) che si occupa del mapping manuale delle entità di dominio sulle tabelle relazionali.

- **Build Automation:** Maven, utilizzato per la gestione del ciclo di vita del software e delle dipendenze.

- **UI Mockup:** JavaFX (FXML) per prototipazione grafica, separata dai flussi principali.

- **Testing Framework:** JUnit 5 (Jupiter), impiegato per la stesura di test unitari (strutturali) e test di integrazione (funzionali).
- **Versioning e IDE:** Git con hosting repository su GitHub. L'ambiente di sviluppo integrato utilizzato è Visual Studio Code (VS Code).
- **Modellazione UML:** PlantUML, integrato direttamente nell'IDE per generare diagrammi come codice (*diagrams-as-code*), facilitando il tracciamento delle modifiche nel versioning.

1.3 Note metodologiche e Uso dell'Intelligenza Artificiale

In linea con le moderne pratiche di Ingegneria del Software e con le direttive del corso, lo sviluppo di *EscapeManager* è stato coadiuvato dall'uso consapevole di strumenti di Intelligenza Artificiale Generativa (GitHub Copilot e Google Gemini).

L'approccio adottato non è stato di delega totale, ma di *AI-Assisted Pair Programming*:

- **Architettura e Design:** Le scelte architetturali (pattern BCE, identificazione dei Design Pattern GoF come State, Strategy e Observer) e la modellazione concettuale sono state guidate da ragionamento umano basato sui principi del corso.
- **Generazione del Codice:** L'IA è stata impiegata come "agente" per la stesura del codice boilerplate (es. implementazione dei DAO, getter/setter, refactoring di massa), per l'impalcatura dei test JUnit e per l'isolamento del database nei test di integrazione, sempre previa fornitura di *prompt* ingegnerizzati e rigorosa code review umana.
- **Documentazione:** L'IA è stata utilizzata come supporto per la formattazione tipografica in LaTeX e per la revisione lessicale del presente documento.
- **Verifica e Controllo:** ogni output generato è stato verificato manualmente, corretto e integrato nel progetto solo se coerente con requisiti e codice effettivo.
- **Limiti d'uso:** nessuna sezione è stata inserita senza revisione critica; la responsabilità finale dei contenuti rimane interamente dell'autore.

1.4 Struttura del documento

Il presente documento è organizzato come segue: il Capitolo 2 presenta l'analisi dei requisiti funzionali e non funzionali; il Capitolo 3 descrive la progettazione architetturale e di dominio; il Capitolo 4 illustra i dettagli implementativi e l'architettura del codice; infine, il Capitolo 5 è dedicato al collaudo e al testing del sistema.

1.5 Limiti del prototipo e sviluppi futuri

Allo stato attuale, *EscapeManager* è un *proof of concept* pienamente funzionale a livello di Business Logic e persistenza dati. L'interazione utente è simulata per semplicità attraverso una Command Line Interface (CLI) o casi di test diretti; i mockup JavaFX sono presenti come prototipo separato. La notifica automatica delle liste d'attesa e la gestione completa delle filiali sono previste come estensioni future. Sviluppi futuri prevedono inoltre l'esposizione di API

REST per un frontend Web/Mobile o, in alternativa, l'implementazione di un'interfaccia grafica desktop completa tramite JavaFX, mantenendo intatto l'attuale Domain Model grazie al rigoroso disaccoppiamento garantito dall'architettura BCE.

2. Analisi dei Requisiti

Per definire i requisiti funzionali del sistema in ottica Object-Oriented, si è proceduto all'identificazione degli attori principali e dei relativi casi d'uso. La modellazione è stata condotta tramite un diagramma dettagliato per l'analisi tecnica.

Il sistema prevede le seguenti figure:

- **Cliente:** Utente finale che interagisce col sistema per consultare il catalogo e prenotare le esperienze.
- **Game Master:** Personale di filiale deputato al controllo operativo delle stanze e delle sessioni.
- **Admin:** Amministratore di sistema incaricato di definire il catalogo e le logiche commerciali.

2.1 Use Case Diagram

La Figura 2.1 rappresenta i casi d'uso effettivi del sistema. Tutti i casi d'uso sono di livello `«user goal»` e sono numerati per mantenere la consistenza con i template. Per mantenere il diagramma ad un alto livello di coesione ed evitare l'anti-pattern della decomposizione funzionale, le operazioni di CRUD standard e l'autenticazione di base sono state omesse o raggruppate. In questa vista viene evidenziata la relazione di estensione (`«extend»`) che modella un flusso alternativo cruciale per il dominio aziendale: la gestione opzionale delle liste d'attesa in caso di esaurimento degli slot disponibili.

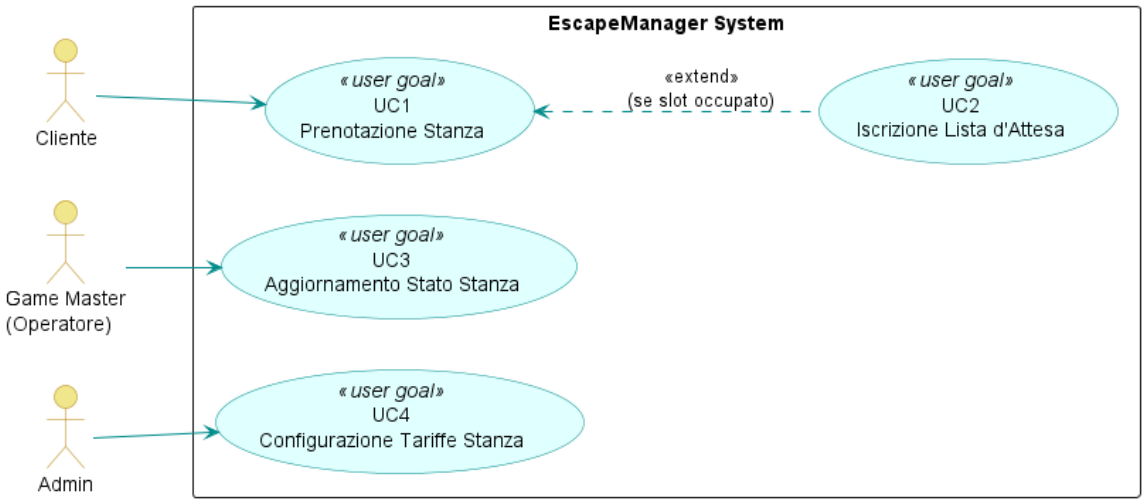

Figura 2.1: Diagramma dei Casi d'Uso

2.1.1 Legenda riferimenti a Mockup e Navigazione

Nei template dei casi d'uso, ogni passo include tra parentesi il riferimento al mockup (**M1..M4**) e al nodo del diagramma di navigazione (**N0..N5**, Figura 2.2).

ID	Mockup associato	Nodo di navigazione
M1	Figura 2.3 (Catalogo e Prenotazione)	N1 Catalogo Stanze, N2 Prenotazione
M2	Figura 2.4 (Lista d'Attesa)	N3 Lista d'Attesa
M3	Figura 2.5 (Stato Stanze)	N4 Aggiorna Stato Stanza
M4	Figura 2.6 (Tariffe)	N5 Configura Tariffe

Tabella 2.1: Mappatura dei riferimenti ai mockup e ai nodi di navigazione

2.2 Use Case Templates

Di seguito vengono dettagliati i template per i casi d'uso primari del sistema, adottando un formato strutturato che esplicita le pre-condizioni, le post-condizioni, il flusso base (Main Success Scenario) e le possibili deviazioni (Alternative Flows). Ogni passo riporta, tra parentesi, i riferimenti a mockup e navigazione (M1..M4, N0..N5). Questa formalizzazione costituisce il contratto per lo sviluppo e la base per la successiva stesura dei test funzionali.

2.2.1 UC1 - Prenotazione Stanza

Nome Use Case	UC1 - Prenotazione Stanza
Attore Principale	Cliente
Livello	User Goal
Scopo	Permettere al cliente di riservare una stanza in uno specifico slot temporale, calcolando la tariffa appropriata.
Pre-condizioni	Il Cliente è autenticato nel sistema. Il sistema ha caricato il catalogo delle stanze.
Post-condizioni	Una nuova Prenotazione viene registrata nel sistema con stato "Confermata". Lo slot orario della stanza risulta occupato.

Flusso Base (Basic Flow)

1. Il Cliente seleziona una Stanza dal catalogo e richiede di visualizzare le disponibilità. (M1, N1)
2. Il Sistema mostra il calendario con le date e gli slot orari disponibili per la Stanza scelta. (M1, N1)
3. Il Cliente seleziona una data, uno slot orario libero e inserisce il numero di giocatori. (M1, N2)
4. Il Sistema calcola il prezzo totale basandosi sulla strategia di prezzo corrente (es. base, weekend) e mostra il riepilogo. (M1, N2)
5. Il Cliente conferma la prenotazione. (M1, N2)
6. Il Sistema registra la prenotazione, aggiorna lo stato dello slot orario e mostra un messaggio di successo. (M1, N2)

Flussi Alternativi (Alternative Flows)

3a. Slot orario non più disponibile (Race condition):

3a.1 Se nel momento della conferma lo slot è stato appena prenotato da un altro utente, il Sistema notifica l'errore.

3a.2 Il Sistema propone al Cliente di tornare al passo 2 o di iscriversi alla Lista d'Attesa (UC2 - *Extend*). (M2, N3)

4a. Numero di giocatori non valido:

4a.1 Se il numero di giocatori inserito supera la capienza massima (o è inferiore alla minima) della Stanza, il Sistema segnala l'errore.

4a.2 Il Cliente è riportato al passo 3 per correggere il dato.

Tabella 2.2: Template del Caso d'Uso: Prenotazione Stanza

2.2.2 UC2 - Iscrizione Lista d'Attesa

Nome Use Case	UC2 - Iscrizione Lista d'Attesa
Attore Principale	Cliente
Livello	User Goal
Scopo	Permettere a un cliente di iscriversi alla lista d'attesa per una stanza attualmente non disponibile, per essere notificato in caso di liberazione.
Pre-condizioni	Il Cliente è autenticato. Lo slot richiesto per la stanza selezionata non è disponibile.
Post-condizioni	Il Cliente è registrato nella tabella `LISTA_ATTESA` per la stanza selezionata.

Flusso Base (Basic Flow)

1. Il Cliente visualizza che lo slot desiderato è occupato. (M2, N3)
2. Il Cliente seleziona l'opzione "Iscriviti alla Lista d'Attesa". (M2, N3)
3. Il Sistema richiede la conferma per l'iscrizione alla lista d'attesa. (M2, N3)
4. Il Cliente conferma. (M2, N3)
5. Il Sistema registra il Cliente nella lista d'attesa e mostra un messaggio di successo. (M2, N3)

Flussi Alternativi (Alternative Flows)

4a. Il Cliente annulla l'iscrizione (M2, N3):

4a.1 Il Cliente seleziona "Annulla" alla richiesta di conferma.

4a.2 Il Sistema interrompe l'operazione e riporta il Cliente al catalogo.

5a. Cliente già in lista (M2, N3):

5a.1 Il Sistema rileva che il Cliente è già iscritto alla lista d'attesa per quella Stanza.

5a.2 Il Sistema notifica l'utente dell'iscrizione già attiva, senza duplicare la registrazione.

2.2.3 UC3 - Aggiornamento Stato Stanza

Nome Use Case	UC3 - Aggiornamento Stato Stanza
Attore Principale	Game Master
Livello	User Goal
Scopo	Aggiornare il ciclo di vita operativo di una stanza (es. da Disponibile a In Corso, da In Corso a In Pulizia).
Pre-condizioni	Il Game Master è autenticato e autorizzato per la sede di competenza.
Post-condizioni	Lo stato della Stanza è aggiornato nel sistema persistente.

Flusso Base (Basic Flow)

1. Il Game Master accede alla dashboard di controllo delle stanze. (M3, N4)
2. Il Sistema mostra l'elenco delle stanze della sede e il loro stato attuale. (M3, N4)
3. Il Game Master seleziona una Stanza in stato "Disponibile" e clicca su "Avvia Sessione". (M3, N4)
4. Il Sistema valida la transizione di stato. (M3, N4)
5. Il Sistema aggiorna lo stato della stanza in "In Corso" e registra l'evento. (M3, N4)

Flussi Alternativi (Alternative Flows)

4a. Transizione di stato non valida (M3, N4):

4a.1 Il Sistema rileva che l'azione richiesta viola il pattern di stato (es. tentativo di avviare una stanza "In Manutenzione").

4a.2 Il Sistema lancia un'eccezione interna e mostra al Game Master un messaggio di errore: "Transizione non consentita dallo stato attuale".

2.2.4 UC4 - Configurazione Tariffe Stanza

Nome Use Case	UC4 - Configurazione Tariffe Stanza
Attore Principale	Admin
Livello	User Goal
Scopo	Assegnare o modificare la strategia di prezzo applicata a una specifica stanza.
Pre-condizioni	L'Admin è autenticato. La Stanza esiste nel sistema.
Post-condizioni	La nuova strategia di prezzo (PricingStrategy) è associata alla Stanza.
Flusso Base (Basic Flow)	
1. L'Admin accede al pannello di amministrazione e seleziona una Stanza. (M4, N5) 2. Il Sistema mostra i dettagli della stanza e l'attuale strategia di prezzo. (M4, N5) 3. L'Admin seleziona l'opzione per modificare la tariffa e sceglie una nuova strategia dall'elenco (es. *Tariffa Base*, *Tariffa Weekend*). (M4, N5) 4. L'Admin salva la configurazione. (M4, N5) 5. Il Sistema aggiorna i dati nel database e conferma il successo dell'operazione. (M4, N5)	

2.3 Requisiti non funzionali

Oltre ai requisiti funzionali, il sistema deve rispettare i seguenti requisiti non funzionali:

- **Prestazioni:** le operazioni principali (ricerca disponibilità, prenotazione, aggiornamento stato) devono completarsi entro 1 secondo su dataset di test (fino a 100 stanze e 1000 prenotazioni).
- **Affidabilità:** le operazioni critiche (es. prenotazione) devono essere atomiche e non lasciare il sistema in stato inconsistente in caso di errore.
- **Usabilità:** la CLI deve guidare l'utente con menu espliciti, validazioni dei dati e messaggi di errore chiari.
- **Manutenibilità:** l'architettura a livelli (CLI, Controllers, DAO, DB) e l'uso di pattern riducono l'accoppiamento e facilitano l'estensione futura.
- **Portabilità:** il sistema deve poter essere eseguito su Windows, macOS e Linux con JDK 25 e PostgreSQL, grazie all'uso di JDBC e configurazioni esterne.

2.4 Diagramma di Navigazione (CLI)

La Figura 2.2 rappresenta il flusso di navigazione della CLI, evidenziando i menu principali e il collegamento diretto con i casi d'uso UC1–UC4. I nodi sono etichettati N0..N5 e sono richiamati nei template dei casi d'uso.

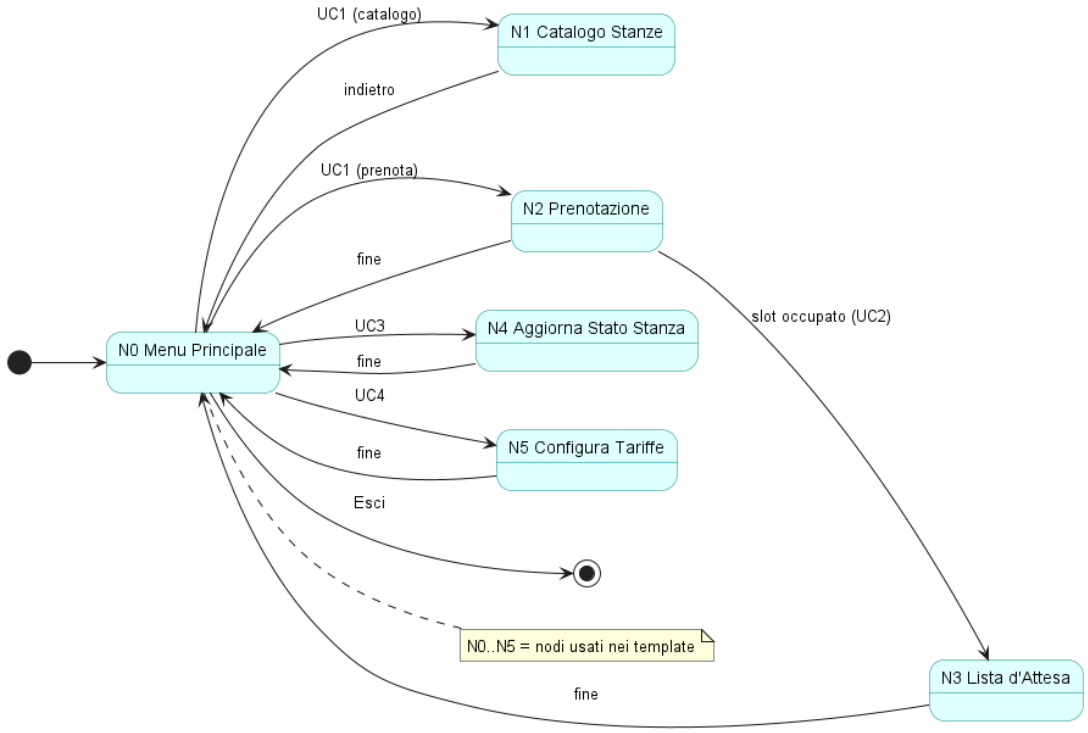

Figura 2.2: Diagramma di navigazione della CLI

2.5 Mockup e Interfaccia Utente (UI)

Al fine di concretizzare l'analisi dei requisiti e validare le interazioni previste nei casi d'uso principali, sono stati realizzati dei mockup per le schermate core del sistema. Questa fase di *prototipazione low-fidelity* garantisce una mappatura visiva diretta tra le intenzioni dell'attore (User Goal) e le effettive funzionalità esposte dal sistema *EscapeManager*. I mockup sono identificati come M1..M4 e richiamati nei template.

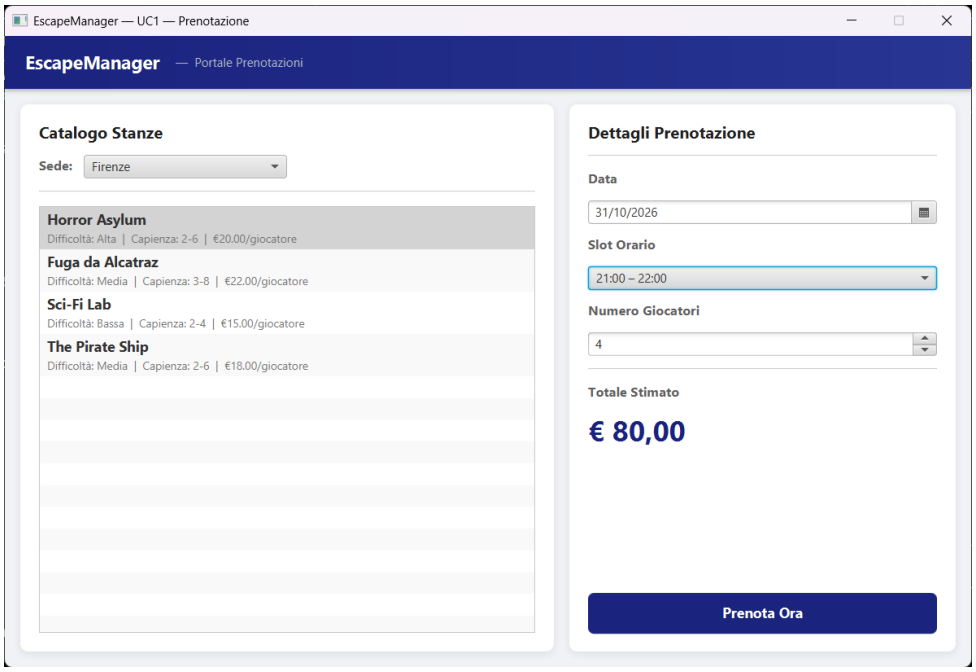

Figura 2.3: [M1] Interfaccia Cliente: Visualizzazione disponibilità e preventivo (UC1)

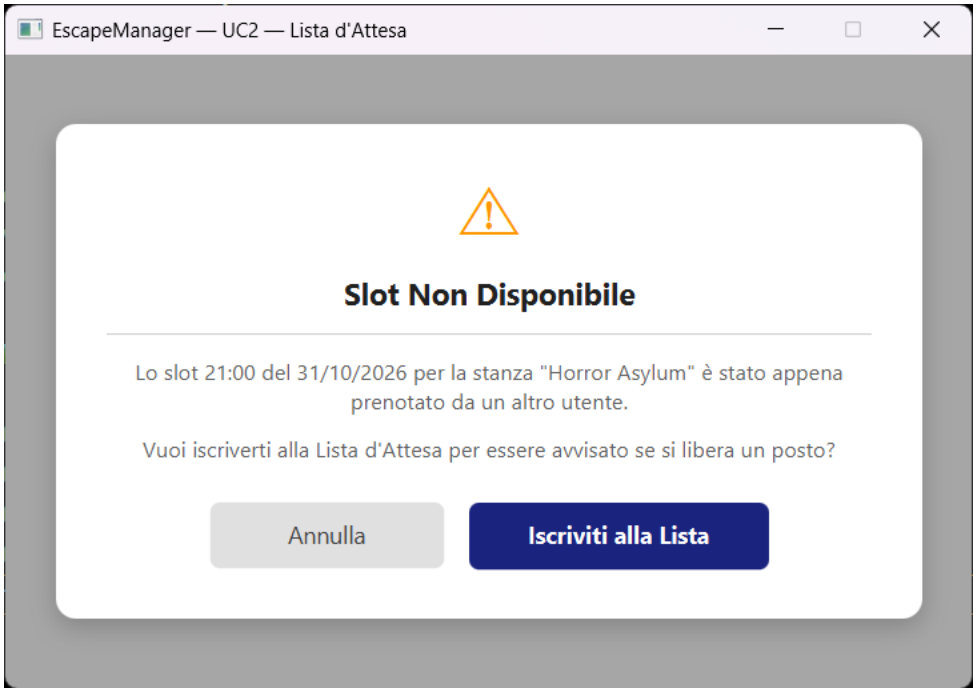

Figura 2.4: [M2] Interfaccia Cliente: Prompt di iscrizione alla Lista d'Attesa (UC2)

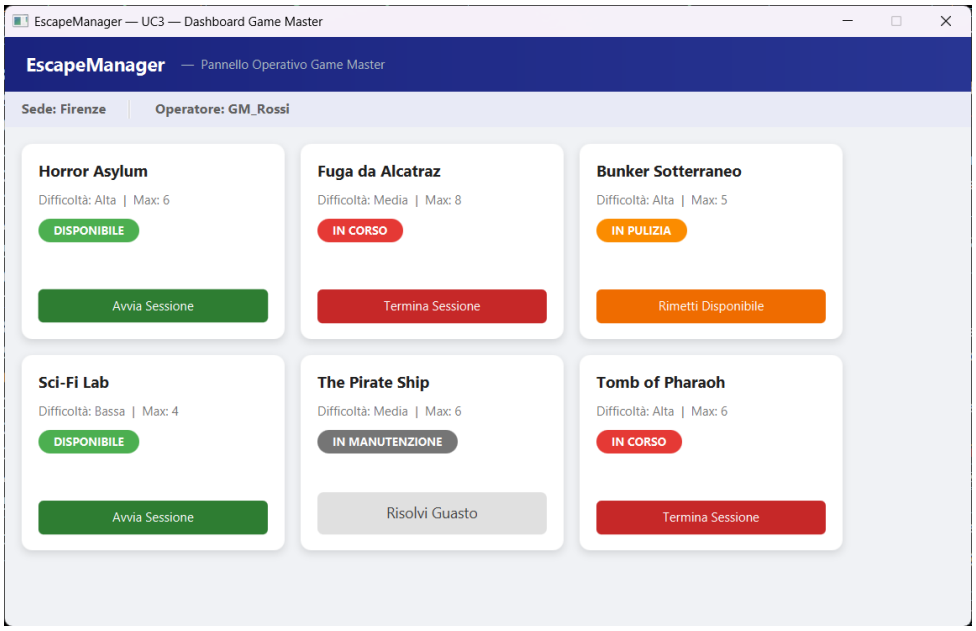

Figura 2.5: [M3] Interfaccia Game Master: Tabellone operativo e controllo stati (UC3)

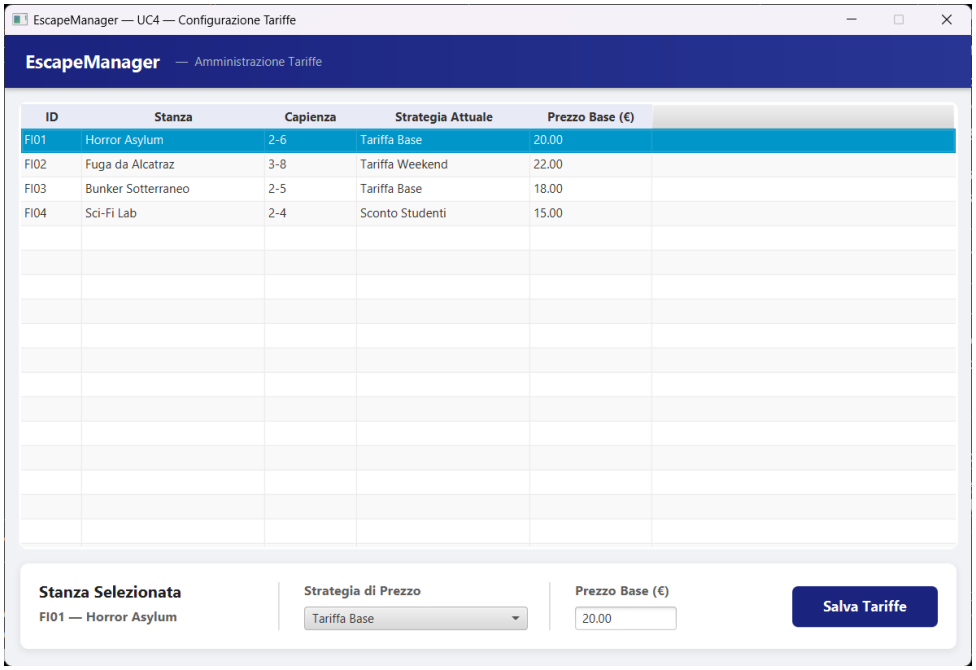

Figura 2.6: [M4] Interfaccia Admin: Configurazione delle strategie di prezzo (UC4)

3. Progettazione Architetturale e di Dominio

Il processo di transizione dall'analisi dei requisiti alla progettazione ha seguito i principi dell'Object-Oriented Design (OOD) e l'approccio BCE (Boundary-Control-Entity). In questo capitolo viene dettagliata la struttura statica del Domain Model, ponendo particolare enfasi sull'applicazione dei Design Pattern fondamentali per garantire la manutenibilità, l'estensibilità e il disaccoppiamento del sistema.

3.1 Architettura a Strati e BCE

L'architettura è organizzata in strati coerenti con il paradigma BCE. Le interazioni tra Boundary (CLI e mockup), Control (Controllers) ed Entity (Domain Model) sono strettamente direzionali, mentre la persistenza è isolata nel layer DAO.

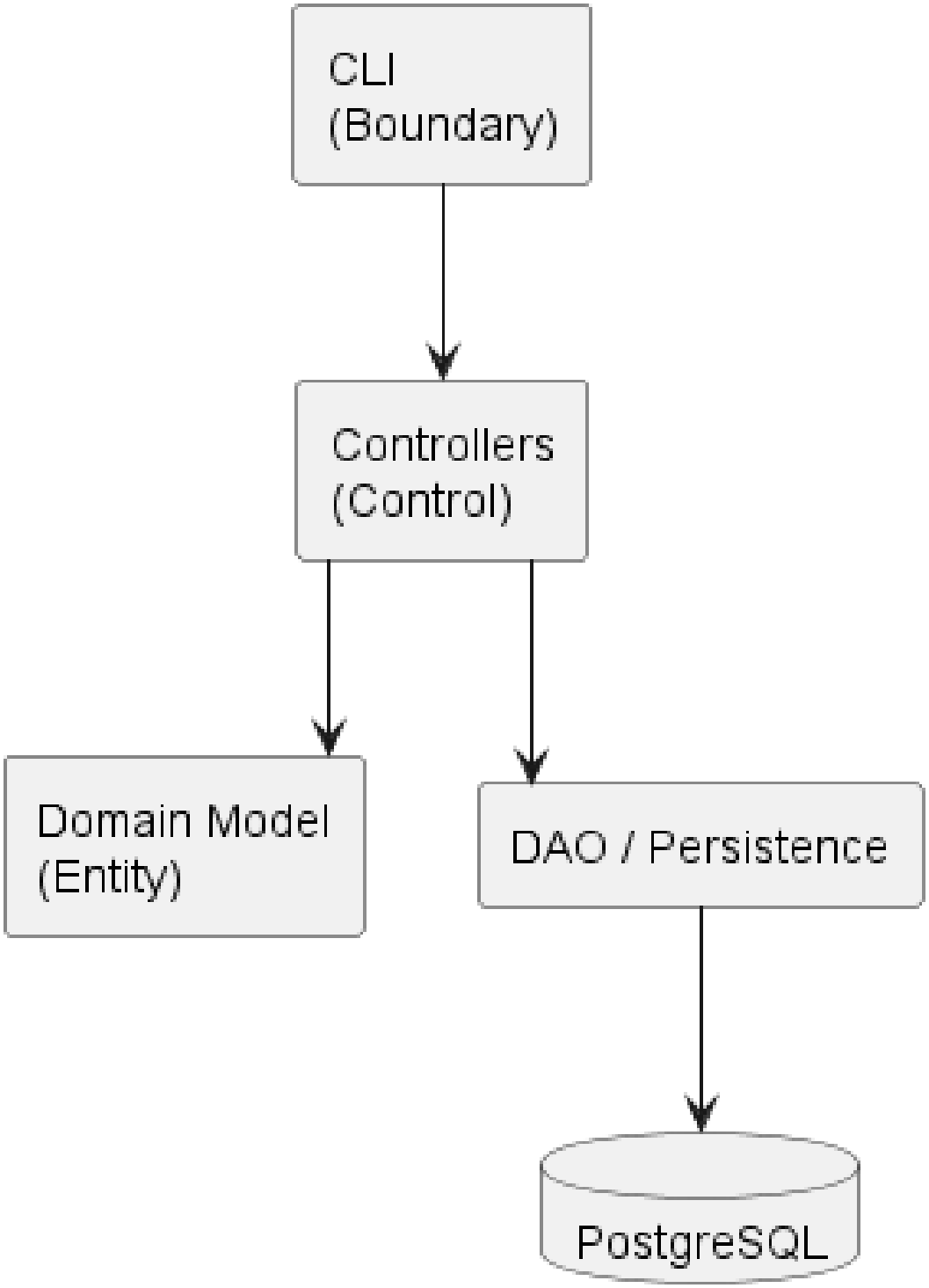

Figura 3.1: Architettura a strati e separazione BCE

3.2 Package Diagram e Dipendenze

Il Package Diagram (Figura 3.2) descrive la struttura logica del progetto e le dipendenze tra i package principali. La CLI accede esclusivamente ai Controllers, i quali orchestrano Domain Model e DAO, mantenendo un flusso di dipendenze unidirezionale.

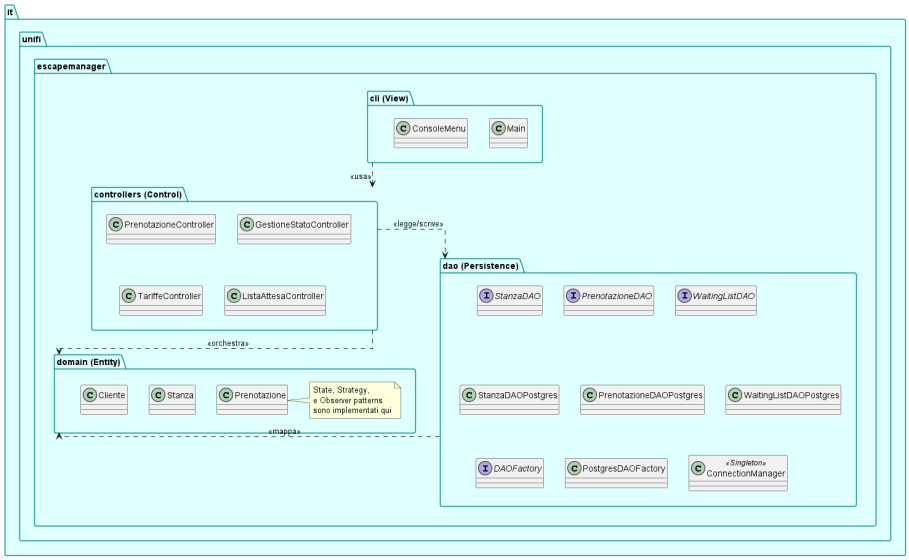

Figura 3.2: Package Diagram dell'architettura di sistema

3.3 Scelte Progettuali e Trade-off

Le principali scelte progettuali sono motivate da requisiti di manutenibilità e chiarezza didattica:

- **DAO + JDBC** invece di ORM: maggiore controllo sul mapping e migliore trasparenza dei flussi dati in un contesto d'esame.
- **CLI come Boundary** per ridurre l'impatto della UI e concentrare lo sforzo sulla logica di business.
- **Pattern GoF espliciti** (State/Strategy/Observer) per rendere evidenti i punti di estensione del dominio.
- **Single Table Inheritance** per la gerarchia `Utente`, semplificando query e autenticazione futura.

3.4 Domain Model (Class Diagram)

La Figura 3.3 illustra il diagramma delle classi relativo al nucleo logico dell'applicativo (Entity layer). Il modello non include volutamente i dettagli legati all'accesso ai dati (DAO) o all'interfaccia utente (View/CLI), concentrandosi esclusivamente sulle regole di business del franchise.

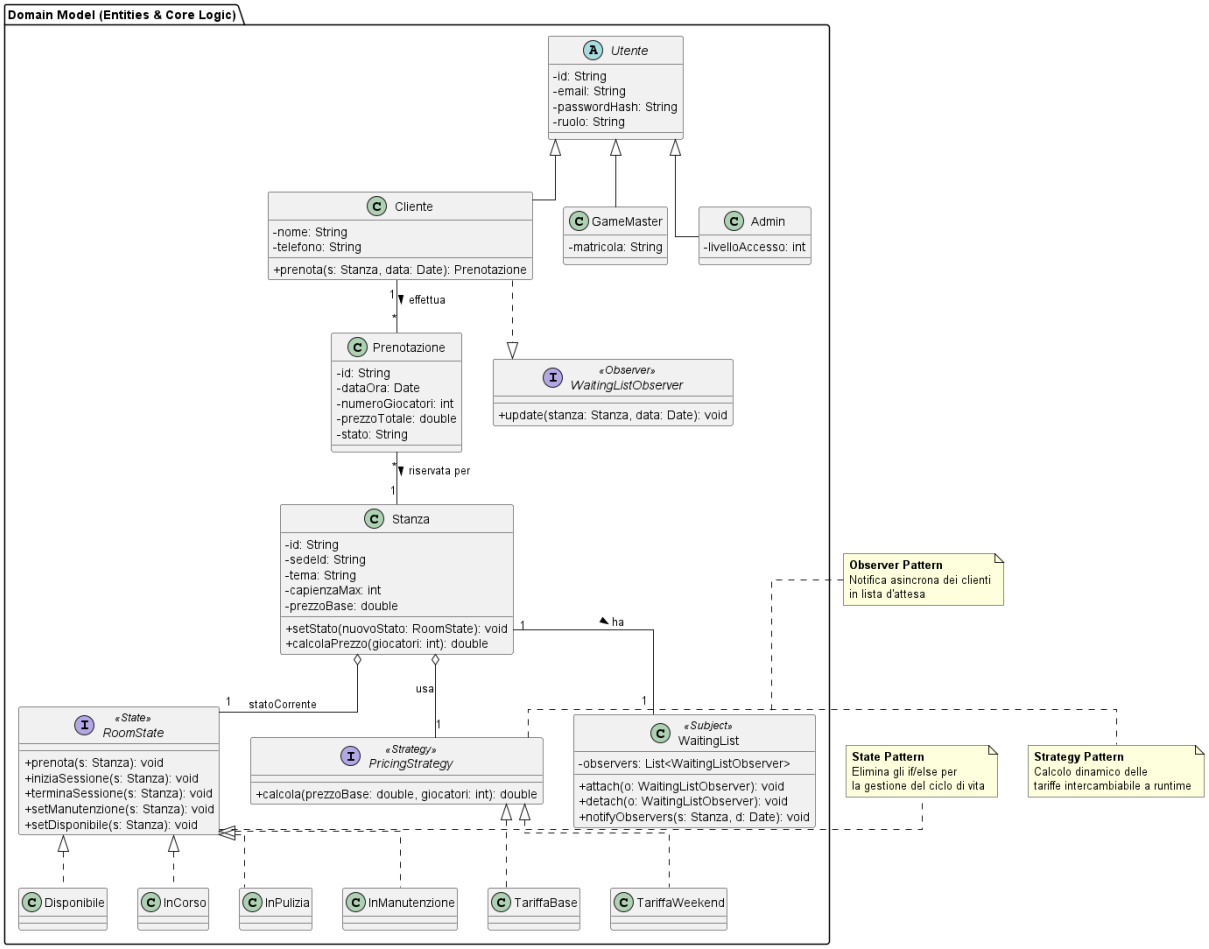

Figura 3.3: Diagramma delle Classi del Domain Model con Design Pattern

3.5 Class Diagram dei Controller e Boundary

Per documentare la vista di implementazione dei layer di controllo e presentazione, la Figura 3.4 mostra le principali classi di coordinamento e le loro dipendenze sui DAO.

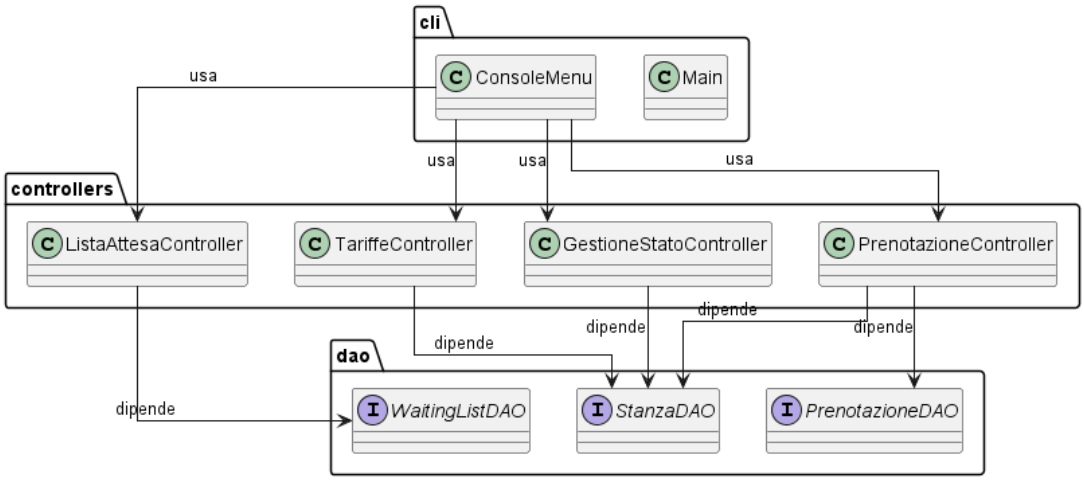

Figura 3.4: Class Diagram: Controllers e Boundary (CLI)

3.5.1 Scelte Architetturali e Design Pattern

L'architettura del dominio è stata concepita per risolvere nativamente le complessità emerse durante l'analisi dei casi d'uso (Capitolo 2), ricorrendo a tre Design Pattern della *Gang of Four (GoF)*:

- **State Pattern (Gestione ciclo di vita):** Come emerso nell'UC3, la `Stanza` attraversa fasi operative rigide (Disponibile, In Corso, In Pulizia, In Manutenzione). Invece di delegare il controllo a complessi blocchi `switch/case` basati su stringhe o enumerazioni, lo stato è stato reificato nell'interfaccia `RoomState`. Questo garantisce il rispetto del principio *Open-Closed*: l'aggiunta di un nuovo stato futuro non richiederà la modifica del codice della classe `Stanza`.

- **Strategy Pattern (Politiche di Prezzo):** Le regole di tariffazione dipendono da fattori esterni (giorno della settimana, festività, tipologia di clienti). La classe `Stanza` delega il calcolo del costo totale all'interfaccia `PricingStrategy`. L'amministratore (UC4) può così iniettare a runtime strategie diverse (es. `TariffaBase` e `TariffaWeekend`) variando l'algoritmo di calcolo in modo trasparente.

- **Observer Pattern (Lista d'Attesa asincrona):** Per soddisfare il requisito dell'UC2 (Iscrizione Lista d'Attesa), è stato predisposto un meccanismo di Publish-Subscribe. La classe `WaitingList` agisce da *Subject*, mantenendo una lista di `WaitingListObserver` (interfaccia implementata dal `Cliente`). L'infrastruttura di iscrizione è implementata e persiste su database; l'invio automatico delle notifiche è considerato un'estensione futura.

Oltre ai pattern, si sottolinea l'uso dell'ereditarietà per la gestione degli attori (`Utente` come classe astratta generalizzata) e la presenza di un riferimento a `Sede` tramite `sede_id` nel modello dati, che consente di mantenere la coerenza con il concetto di franchising pur non essendo oggetto di operazioni dirette nella CLI.

3.5.2 State Machine Diagram

Per documentare formalmente il ciclo di vita operativo della Stanza, è stato prodotto il diagramma a stati della Figura 3.5. Questo diagramma è una vista complementare al class diagram del Domain Model e descrive le transizioni valide tra i 4 stati concreti dell'interfaccia `RoomState`.

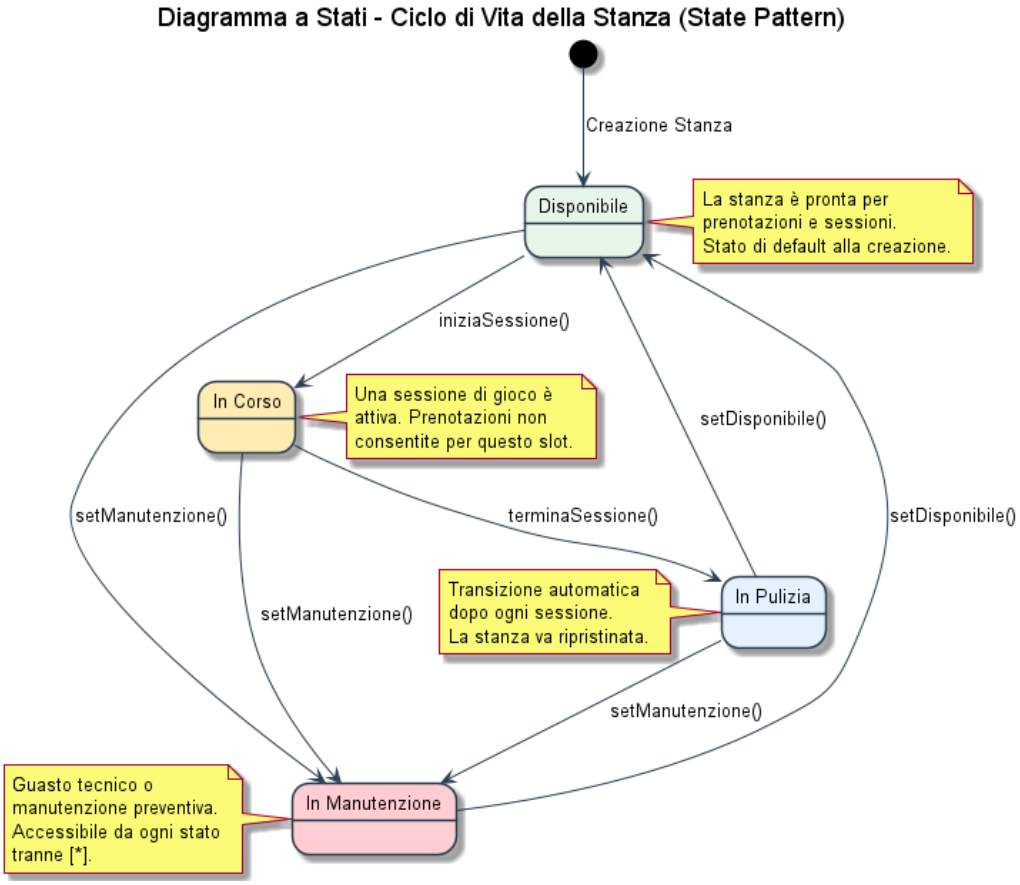

Figura 3.5: State Machine Diagram per il ciclo di vita della Stanza (RoomState)

3.6 Progettazione del Database

Per garantire la persistenza dei dati e supportare l'infrastruttura di un sistema centralizzato, il Domain Model Object-Oriented è stato mappato su un database relazionale (PostgreSQL). La transizione dal modello concettuale allo schema logico ha richiesto l'applicazione di specifiche strategie di *Object-Relational Mapping (ORM)*.

3.6.1 Diagramma Entity-Relationship

La Figura 3.6 mostra il diagramma Entità-Relazione (ER) del database, illustrando le tabelle principali, le chiavi primarie (PK) e i vincoli di chiave esterna (FK).

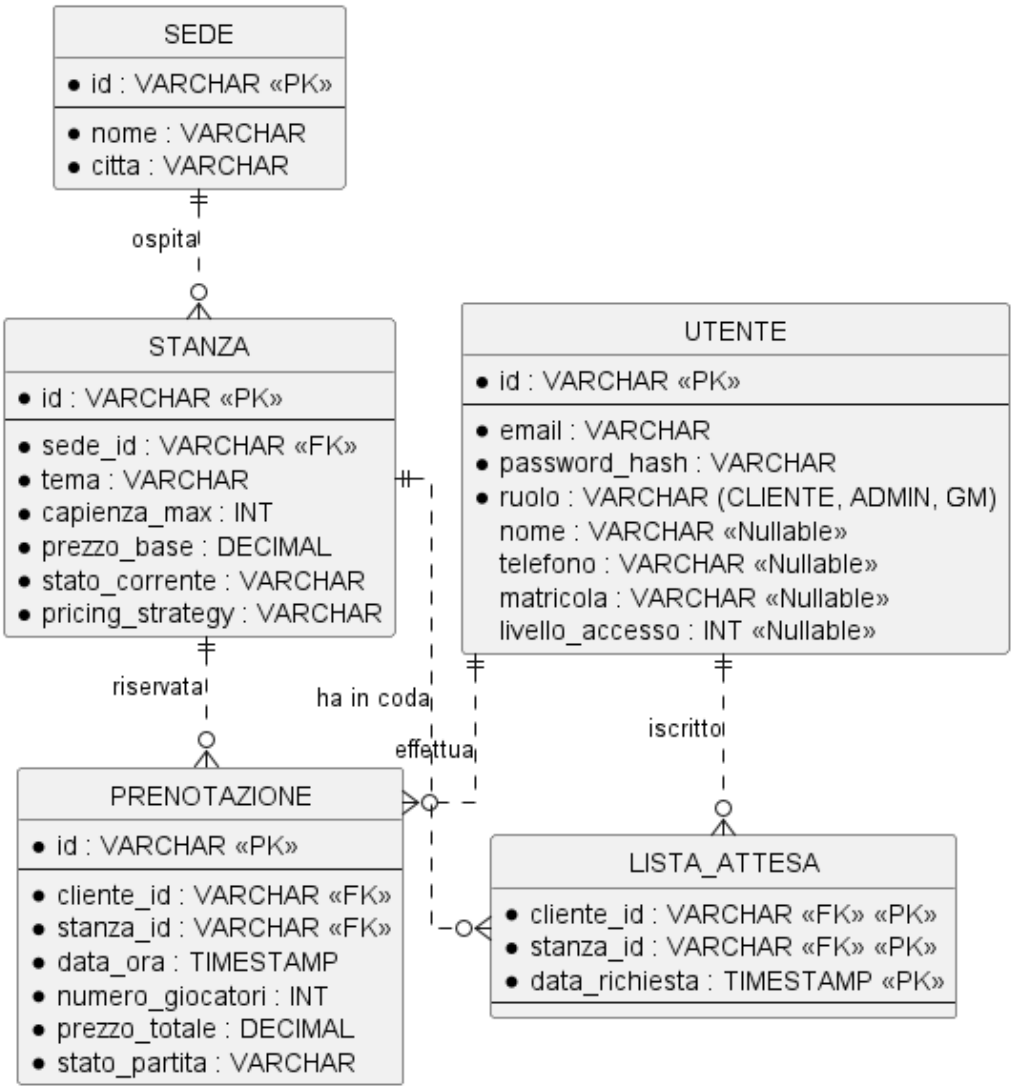

Figura 3.6: Diagramma Entity-Relationship (ER) del sistema

3.6.2 Strategie di Mapping e Modello Logico

Il design relazionale affronta la dicotomia tra il paradigma a oggetti e quello tabellare attraverso le seguenti scelte implementative:

- **Mapping dell'Ereditarietà (Single Table Strategy):** La gerarchia della classe astratta `Utente` (Cliente, GameMaster, Admin) è stata mappata utilizzando il pattern *Single Table Inheritance*. È stata creata un'unica tabella `UTENTE` provvista di una colonna discriminante (`ruolo`). Gli attributi specifici delle sottoclassi (es. `matricola` per il Game Master o `telefono` per il Cliente) sono stati resi *nullable*. Questa scelta ottimizza le query di autenticazione evitando costose operazioni di `JOIN`.

- **Mapping dei Design Pattern (State e Strategy):** Le interfacce `RoomState` e `PricingStrategy` non diventano tabelle separate. Nella tabella `STANZA` sono persistite tramite stringhe (ad es. `stato_corrente=IN_MANUTENZIONE`). A livello di codice Java, i Data Access Object (DAO) utilizzano un meccanismo di *Factory* per istanziare l'oggetto concreto corrispondente al valore letto dal database.

- **Associazioni Molti-a-Molti (Observer Pattern):** La logica Publish-Subscribe della Lista d'Attesa (UC2) si traduce relazionalmente nella tabella di *junction* `LISTA_ATTESA`. Questa tabella mappa la relazione N:N tra Clienti e Stanze, utilizzando una chiave primaria composita formata dalle foreign key e dal timestamp della richiesta.

Di seguito viene riportato lo **Schema Logico Relazionale** formalizzato:

- **SEDE**(id, nome, citta)
- **STANZA**(id, sede_id, tema, capienza_max, prezzo_base, stato_corrente, pricing_strategy)
- **UTENTE**(id, email, password_hash, ruolo, nome*, telefono*, matricola*, livello_accesso*) — * *attributo nullable a seconda del ruolo*
- **PRENOTAZIONE**(id, cliente_id, stanza_id, data_ora, numero_giocatori, prezzo_totale, stato_partita)
- **LISTA_ATTESA**(cliente_id, stanza_id, data_richiesta)

3.7 Dinamica del Sistema (Implementation Perspective)

Per illustrare le dinamiche di interazione a *run-time* tra gli oggetti, si fa riferimento all'approccio descritto nello standard UML per la *Implementation Perspective*. Di seguito vengono presentati due Sequence Diagram per i casi d'uso più rappresentativi.

3.7.1 UC1 - Prenotazione Stanza

La Figura 3.7 mostra il Sequence Diagram relativo al caso d'uso principale. Il diagramma evidenzia il rispetto del pattern architetturale BCE: l'interfaccia utente (Boundary) delega l'elaborazione al `PrenotazioneController` (Control), il quale orchestra il reperimento delle risorse persistenti e demanda la logica matematica di calcolo del prezzo all'entità `Stanza` (Entity). Particolare attenzione è stata posta nel disaccoppiamento del Controller dall'accesso ai dati, delegato interamente a un pattern creazionale.

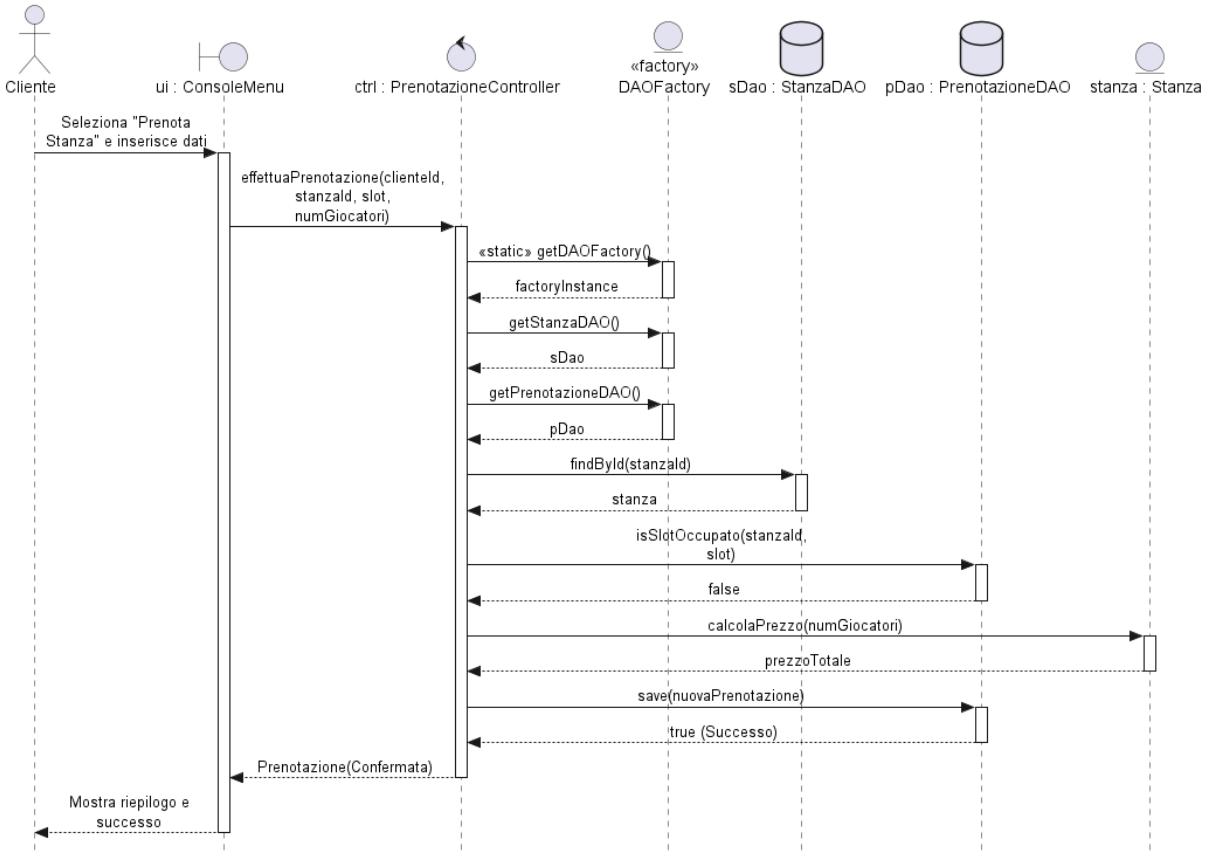

Figura 3.7: Sequence Diagram del caso d'uso UC1 (Prenotazione Stanza)

3.7.2 UC4 - Configurazione Tariffe

La Figura 3.8 descrive il flusso di configurazione delle tariffe. L'Admin opera tramite la CLI, la quale delega al `TariffeController` la selezione della strategia e l'aggiornamento della stanza via DAO.

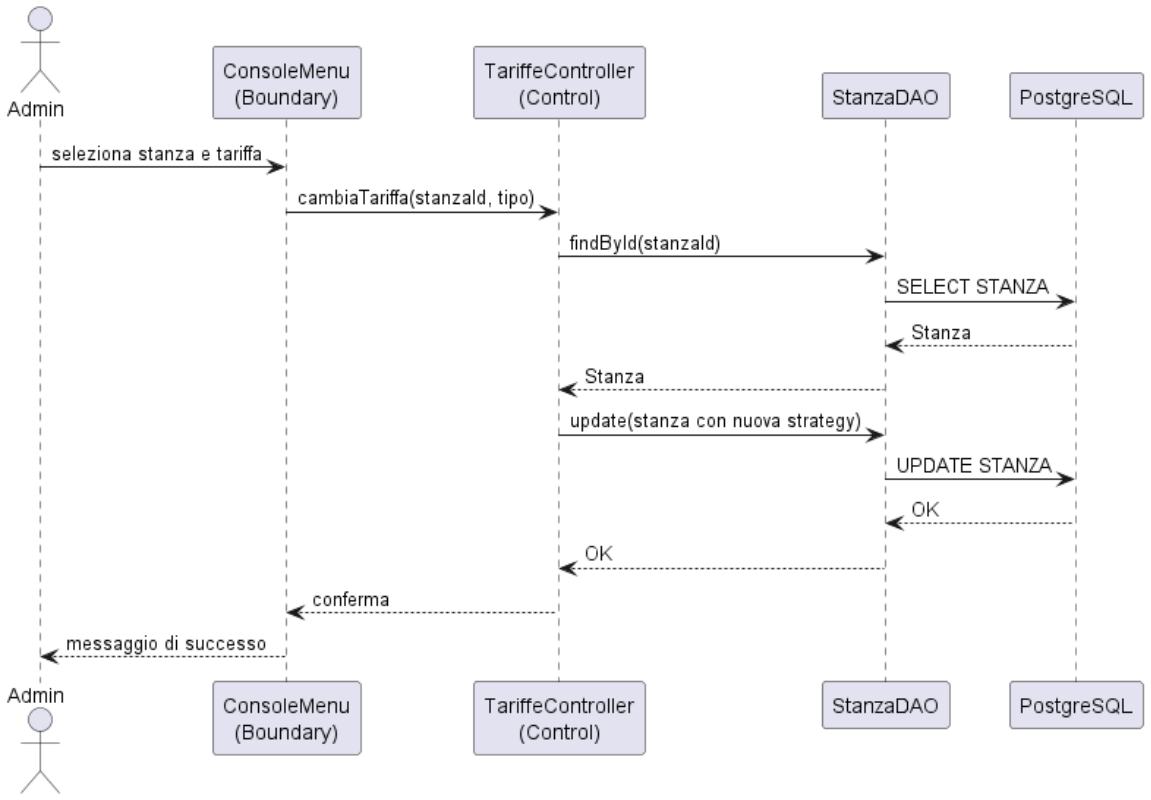

Figura 3.8: Sequence Diagram del caso d'uso UC4 (Configurazione Tariffe)

3.8 Pattern Creazionali: DAO Factory

Come evincibile dal diagramma di sequenza, il sistema adotta il pattern **Abstract Factory** (nella specifica accezione di *DAO Factory*) per la generazione degli oggetti preposti all'accesso ai dati. L'obiettivo di questa scelta progettuale è garantire il totale disaccoppiamento tra la Business Logic (Controller) e l'infrastruttura di persistenza (PostgreSQL). Il Controller non istanzia mai in modo diretto una classe concreta come `StanzaDAOPostgres`, ma si affida all'interfaccia esposta dalla Factory:

```
// Reperimento dinamico dei DAO indipendentemente dal DB sottostante
DAOFactory factory = DAOFactory.getDAOFactory();
StanzaDAO stanzaDao = factory.getStanzaDAO();
PrenotazioneDAO prenotazioneDao = factory.getPrenotazioneDAO();
```

Listing 3.1: Utilizzo del pattern DAO Factory nel Controller

Questa architettura rende il sistema altamente manutenibile ed estensibile: la migrazione verso un diverso DBMS (es. MySQL o un DB NoSQL) richiederebbe unicamente l'implementazione di una nuova `Factory` concreta, lasciando intatto il Domain Model e i Controller.

4. Implementazione e Architettura del Codice

Questo capitolo descrive la traduzione del Domain Model e dello Schema Logico in codice sorgente Java. L'architettura software è stata strutturata seguendo il pattern architetturale *Layered* (a strati), che ricalca logicamente il paradigma BCE (Boundary-Control-Entity), garantendo un elevato disaccoppiamento e facilitando il testing isolato dei componenti.

4.1 Architettura dei Package

L'applicazione è organizzata in quattro package principali (Figura 3.2). Le dipendenze tra i package sono rigorosamente direzionali: lo strato di presentazione (`cli`) non accede mai direttamente al database, ma passa per i `controllers`, che a loro volta orchestrano il `domain` e si interfacciano con i `dao`.

4.1.1 cli (Boundary)

Contiene il punto di ingresso dell'applicazione e la CLI. La classe `ConsoleMenu` gestisce l'interazione con l'utente, la validazione di input di base e la delega delle operazioni ai Controller.

4.1.2 controllers (Control)

Raccoglie i Controller che implementano i flussi dei casi d'uso. Ogni Controller incapsula la logica di coordinamento, chiama i DAO e applica le regole di business esposte dal Domain Model.

4.1.3 domain (Entity)

Include le entità di dominio (`Stanza`, `Prenotazione`, `Utente` e sottoclassi) e l'implementazione dei Design Pattern principali (State, Strategy, Observer).

4.1.4 dao (Persistence)

Contiene le interfacce DAO, le implementazioni PostgreSQL e l'oggetto `ConnectionManager`. L'obiettivo è isolare la persistenza e mantenere il dominio indipendente dalla tecnologia di storage.

4.2 Livello di Persistenza (Pattern DAO)

Per soddisfare i requisiti del modulo di Laboratorio di Informatica, l'accesso al database PostgreSQL è stato isolato tramite il pattern **Data Access Object (DAO)**. Questo garantisce che la logica di business rimanga del tutto agnostica rispetto alla tecnologia di persistenza.

La connessione al database è gestita tramite un **Singleton Pattern** (`ConnectionManager`), che assicura l'esistenza di un'unica istanza di connessione (o di un connection pool) condivisa per tutta l'applicazione, evitando l'overhead di connessioni multiple.

```java
public interface GenericDAO<T, ID> {
    void save(T entity);
    T findById(ID id);
    List<T> findAll();
    void update(T entity);
    void delete(ID id);
}

public interface StanzaDAO extends GenericDAO<Stanza, String> {
    List<Stanza> findBySedeAndStato(String sedeId, String
        statoCorrente);
}
```

Listing 4.1: Interfaccia DAO e uso dei Generics

In modo analogo, `PrenotazioneDAO` e `WaitingListDAO` incapsulano le operazioni di persistenza per prenotazioni e liste d'attesa.

4.3 Configurazione della Connessione al Database

Le credenziali di accesso al database non sono hardcoded: vengono lette da `db.properties` (in `src/main/resources`) o da variabili d'ambiente, separando configurazione e codice ed evitando la pubblicazione accidentale di segreti.

```java
Properties props = loadProperties();
String url = firstNonBlank(
    System.getenv("EM_DB_URL"),
    System.getProperty("db.url"),
    props.getProperty("db.url"),
    "jdbc:postgresql://localhost:5432/escapemanager"
);
```

Listing 4.2: Caricamento della configurazione DB dal classpath

4.4 Validazione Input e Gestione Errori

La CLI valida i formati di input (numeri, date e orari) prima di invocare i Controller. Le eccezioni di business (`ValidationException` e sottoclassi) propagano errori gestibili dalla UI, mantenendo la logica di validazione separata dalla presentazione.

4.5 Implementazione dei Design Pattern

Di seguito vengono riportati i frammenti di codice più significativi che mostrano l'implementazione pratica dei Design Pattern progettati nel Capitolo 3.

4.5.1 State Pattern

Lo State Pattern elimina la necessità di costrutti condizionali complessi per la gestione del ciclo di vita della `Stanza`. Ogni stato concreto implementa il comportamento specifico e gestisce, ove necessario, la transizione allo stato successivo.

```java
public interface RoomState {
    void prenota(Stanza stanza);
    void iniziaSessione(Stanza stanza);
    void terminaSessione(Stanza stanza);
    void setManutenzione(Stanza stanza);
    void setDisponibile(Stanza stanza);
    String getStatusString();
}

public class DisponibileState implements RoomState {
    @Override
    public void iniziaSessione(Stanza stanza) {
        stanza.setStato(new InCorsoState()); // Transizione di stato
    }

    @Override
    public void terminaSessione(Stanza stanza) {
        throw new IllegalStateException("Impossibile terminare una
           sessione per una stanza DISPONIBILE.");
    }
    // ... altri metodi
}
```

Listing 4.3: Implementazione dello State Pattern

4.5.2 Strategy Pattern

Il calcolo del prezzo della prenotazione (UC1) è demandato alla `PricingStrategy`. L'Admin, tramite il `TariffeController`, seleziona la strategia appropriata e la imposta sulla stanza.

```java
public interface PricingStrategy {
    double calcola(double prezzoBase, int numeroGiocatori);
}

public class TariffaWeekend implements PricingStrategy {
    private static final double SOVRAPPREZZO = 1.20; // +20%

    @Override
    public double calcola(double prezzoBase, int numeroGiocatori) {
        return (prezzoBase * numeroGiocatori) * SOVRAPPREZZO;
    }
}
```

Listing 4.4: Implementazione dello Strategy Pattern

4.5.3 Observer Pattern

La lista d'attesa (UC2) implementa un sistema Publisher-Subscriber puro in Java, disaccoppiando l'evento di cancellazione di una prenotazione dall'invio effettivo delle notifiche.

```java
public interface WaitingListObserver {
    void update(Stanza stanza, LocalDateTime slotLiberato);
}

public class WaitingList {
    private List<WaitingListObserver> observers = new ArrayList<>();

    public void attach(WaitingListObserver observer) {
        observers.add(observer);
    }

    public void notifyObservers(Stanza stanza, LocalDateTime slot) {
        for (WaitingListObserver obs : observers) {
            obs.update(stanza, slot);
        }
    }
}
```

Listing 4.5: Implementazione dell'Observer Pattern

4.5.4 Builder Pattern

La classe `Prenotazione` necessita di 7 parametri per la sua costruzione. Per migliorare la leggibilità e prevenire errori nell'ordine dei parametri, è stato implementato il pattern **Builder** (GoF — Creazionale). Questo pattern è particolarmente utile quando una classe ha molti attributi opzionali o quando la costruzione dell'oggetto richiede più passi logici.

```java
Prenotazione p = new Prenotazione.Builder()
        .id(UUID.randomUUID().toString())
        .clienteId("C01")
        .stanzaId("R01")
        .dataOra(LocalDateTime.of(2026, 10, 31, 21, 0))
        .numeroGiocatori(4)
        .prezzoTotale(80.0)
        .statoPartita("CONFERMATA")
        .build();
```

Listing 4.6: Builder Pattern per la classe Prenotazione

4.6 Dependency Injection e Testabilità

I Controller ricevono i DAO tramite costruttore, facilitando la sostituzione delle dipendenze in fase di test. La CLI si limita a comporre l'oggetto e a delegare, mantenendo separati input/output e logica di business.

```java
PrenotazioneDAO prenotazioneDAO = factory.getPrenotazioneDAO();
StanzaDAO stanzaDAO = factory.getStanzaDAO();
```

```
PrenotazioneController controller = new PrenotazioneController(
    prenotazioneDAO, stanzaDAO);
```

Listing 4.7: Iniezione delle dipendenze nel controller

4.7 Gestione Centralizzata delle Eccezioni (Information Hiding)

Un aspetto cruciale dell'implementazione strutturata ha riguardato la gestione dei flussi alternativi e degli errori. Invece di affidarsi alle generiche eccezioni standard di Java (es. `RuntimeException`), è stata disegnata una gerarchia di eccezioni custom radicata nella classe `EscapeManagerException`.

Come illustrato nella Figura 4.1, la gerarchia divide formalmente gli errori in base al layer architetturale in cui si verificano:

- **`ValidationException`:** utilizzata dal dominio per segnalare violazioni delle invarianti di business (es. la sottoclasse `SlotNonDisponibileException`).
- **`DatabaseException`:** utilizzata dal layer DAO per incapsulare problemi di persistenza.

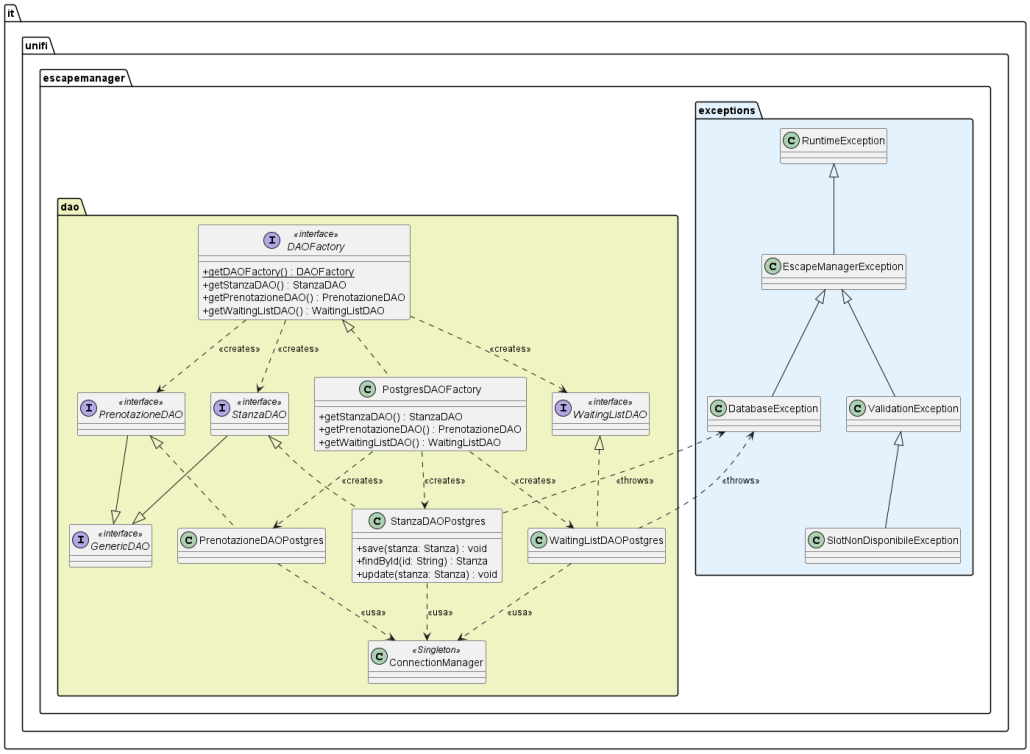

Figura 4.1: Diagramma delle Classi: Pattern DAO Factory e Gerarchia Eccezioni

Questa struttura risponde in modo rigoroso al principio dell'**Information Hiding**. Nel livello DAO, le `SQLException` lanciate dal driver JDBC vengono intercettate, incapsulate in una `DatabaseException` e rilanciate. In questo modo, eventuali stack trace legati a driver SQL o dettagli implementativi del database non inquinano mai i livelli superiori (Boundary e Control).

```java
try (PreparedStatement stmt = conn.prepareStatement(query)) {
    // Esecuzione query...
} catch (SQLException e) {
    // La SQLException originaria viene nascosta ai livelli superiori
    throw new DatabaseException("Errore di accesso ai dati durante il
        salvataggio.", e);
}
```

Listing 4.8: Incapsulamento delle eccezioni SQL nel layer DAO

5. Collaudo e Testing

La fase di verifica e validazione del sistema **EscapeManager** è stata condotta seguendo un approccio multilivello, utilizzando i framework **JUnit 5** e **Mockito**. L'obiettivo del collaudo non è stato solo quello di garantire l'assenza di difetti evidenti (bug), ma soprattutto di dimostrare la robustezza dell'architettura e la perfetta aderenza ai requisiti funzionali delineati nel Capitolo 2.

5.1 Strategia di Verifica

La strategia di test è organizzata su quattro livelli complementari:

- **Unit Test (White-Box):** verificano le logiche pure del Domain Model e dei Design Pattern in completo isolamento.
- **Mock Test (Mockito):** testano i Controller sostituendo i DAO reali con oggetti simulati, isolando la logica di business dal database.
- **Integration Test (DAO/DB):** verificano il mapping ORM e la correttezza delle query principali su database reale.
- **Functional Test (Black-Box):** esercitano i Controller come surrogati della UI, in linea con i template UC.

5.2 Setup e Dati di Test

Per garantire test ripetibili, prima di ogni test di integrazione o funzionale il database viene riportato ad uno stato noto tramite `DatabaseHelper.resetTestDatabase()`, che esegue TRUNCATE e reinserisce dati minimi consistenti (una stanza e due clienti).

5.3 Testing Strutturale (White-Box)

I test strutturali (o Unit Test) si sono concentrati sulle classi del `Domain Model`, con l'obiettivo di massimizzare la copertura (Code Coverage) dei rami condizionali e verificare la correttezza algoritmica, in totale isolamento dalle dipendenze esterne.

Particolare attenzione è stata dedicata al collaudo dei Design Pattern. Il Listing 5.1 mostra un test parametrizzato per la verifica delle politiche di prezzo dinamiche (*Strategy Pattern*).

```
class PricingStrategyTest {

    @Test
    void testTariffaBaseCalcoloCorretto() {
        // Setup
        Stanza stanza = new Stanza("R01", "FI01", "Horror Asylum", 6,
            20.0);
        stanza.setPricingStrategy(new TariffaBase());

        // Execution: 4 giocatori a 20 euro base = 80 euro
```

```
        double prezzoCalcolato = stanza.calcolaPrezzo(4);

        // Assertion
        assertEquals(80.0, prezzoCalcolato);
    }
}
```

Listing 5.1: Unit Test per la logica di calcolo del prezzo (Strategy Pattern)

Oltre al calcolo del prezzo, i test unitari verificano anche le transizioni di stato della `Stanza` (State Pattern, con 11 test per tutte le transizioni valide e illegali), il meccanismo Observer della `WaitingList` (attach, detach, notify), e la costruzione fluente tramite `Builder Pattern`.

5.4 Testing con Mock (Mockito)

Per verificare la correttezza della logica di business dei Controller *indipendentemente* dal database, sono stati implementati test con **Mockito**. Questo framework permette di creare oggetti simulati (mock) delle interfacce DAO, controllando il loro comportamento e verificando le interazioni.

L'annotazione `@ExtendWith(MockitoExtension.class)` abilita l'integrazione automatica con JUnit 5, mentre `@Mock` inietta i DAO simulati e `verify()` asserisce che i metodi attesi siano stati invocati.

```
@ExtendWith(MockitoExtension.class)
class PrenotazioneControllerMockTest {

    @Mock private PrenotazioneDAO prenotazioneDAO;
    @Mock private StanzaDAO stanzaDAO;
    private PrenotazioneController controller;

    @BeforeEach
    void setUp() {
        controller = new PrenotazioneController(prenotazioneDAO,
            stanzaDAO);
    }

    @Test
    void testPrenotazioneRiuscita_FlussoBase_UC1() {
        Stanza stanza = new Stanza("R01", "FI01", "Horror", 6, 20.0);
        when(stanzaDAO.findById("R01")).thenReturn(stanza);
        when(prenotazioneDAO.isSlotOccupato(eq("R01"), any())).
            thenReturn(false);

        Prenotazione result = controller.effettuaPrenotazione("C01",
            "R01",
                LocalDateTime.of(2026, 10, 31, 21, 0), 4);

        assertNotNull(result);
        assertEquals(80.0, result.getPrezzoTotale());
        verify(prenotazioneDAO, times(1)).save(any(Prenotazione.class
            ));
```

```
    }
}
```

Listing 5.2: Mock Test: flusso base UC1 con Mockito

5.5 Testing di Integrazione (DAO e Database)

Per verificare il corretto interfacciamento tra l'applicazione Java e il database PostgreSQL (livello ORM), sono stati implementati test di integrazione sulle classi DAO. Per garantire la **ripetibilità** e l'isolamento, il costrutto `@BeforeEach` invoca `DatabaseHelper.resetTestDatabase()`, che esegue TRUNCATE e reinserisce un dataset minimo prima di ogni metodo di test.

```java
class StanzaDAOPostgresTest {
    private StanzaDAO stanzaDao;

    @BeforeEach
    void setUp() {
        DatabaseHelper.resetTestDatabase();
        stanzaDao = DAOFactory.getDAOFactory().getStanzaDAO();
    }

    @Test
    void testSaveAndFindById() {
        Stanza stanza = stanzaDao.findById("R01");
        assertNotNull(stanza);
        assertEquals("Horror Asylum", stanza.getTema());
    }
}
```

Listing 5.3: Integration Test per StanzaDAOPostgres con reset dello stato

5.6 Testing Funzionale (Black-Box e Tracciabilità)

Il livello più alto di collaudo ha riguardato i `Controllers`. In questa fase (Black-Box Testing) la logica interna viene ignorata e il sistema viene testato esclusivamente in base agli input e agli output attesi.

Per garantire la massima **tracciabilità**, ogni test funzionale è stato mappato direttamente su uno specifico *Use Case Template* (Sezione 2.2). Sono stati collaudati sia i *Flussi Base* che i *Flussi Alternativi* (le eccezioni).

5.6.1 Catalogo dei Test

La Tabella 5.1 riassume l'intera suite di test, indicando tipo, classe e identificativo.

ID	Tipo	Test
U1	**Unit**	`StanzaTest.` `testTransizioneCorretta`
U2	**Unit**	`StanzaTest.` `testTransizioneInCorso_TerminaSessione_VaInPulizia`
U3	**Unit**	`StanzaTest.` `testTransizioneDisponibile_ImpostaManutenzione`
U4	**Unit**	`StanzaTest.` `testCicloCompletoStati`
U5	**Unit**	`StanzaTest.` `testEccezioneStatoNonValido (+ 3 varianti)`
U6	**Unit**	`PricingStrategyTest.` `testTariffaBaseCalcoloCorretto`
U7	**Unit**	`PricingStrategyTest.` `testTariffaWeekendMaggiorazione`
U8	**Unit**	`PricingStrategyTest.` `testCambioStrategiaARuntimeDaBaseAWeekend`
U9	**Unit**	`WaitingListObserverTest.` `testAttachENotifica`
U10	**Unit**	`WaitingListObserverTest.` `testDetachNonRiceveNotifica`
U11	**Unit**	`WaitingListObserverTest.` `testNotificaMultipleObservers`
U12	**Unit**	`PrenotazioneBuilderTest.` `testBuilderCreaPrenotazioneCompleta`
M1	**Mock**	`PrenotazioneControllerMockTest.` `testPrenotazioneRiuscita_FlussoBase_UC1`
M2	**Mock**	`PrenotazioneControllerMockTest.` `testPrenotazioneNegata_SlotOccupato_UC1_Alt3a`
M3	**Mock**	`PrenotazioneControllerMockTest.` `testPrenotazioneNegata_CapienzaSuperata_UC1_Alt4a`
M4	**Mock**	`GestioneStatoControllerMockTest.` `testAvviaSessione_DaDisponibile_UC3_FlussoBase`
M5	**Mock**	`GestioneStatoControllerMockTest.` `testTransizioneNonValida_UC3_Alt4a`
M6	**Mock**	`ListaAttesaControllerMockTest.` `testIscrizioneRiuscita_UC2_FlussoBase`
M7	**Mock**	`ListaAttesaControllerMockTest.` `testIscrizioneDuplicata_UC2_Alt5a`
M8	**Mock**	`TariffeControllerMockTest.` `testCambioTariffaWeekend_UC4_FlussoBase`
I1	**Integration**	`StanzaDAOPostgresTest.` `testSaveAndFindById`
F1	**Functional**	`PrenotazioneControllerTest.` `testPrenotazioneNegataPerSlotOccupato_UC1_Alt3a`
F2	**Functional**	`ListaAttesaControllerTest.` `testIscrizioneListaAttesa_OK`
F3	**Functional**	`ListaAttesaControllerTest.` `testIscrizioneListaAttesa_Duplicato`
F4	**Functional**	`GestioneStatoControllerTest.` `testEseguiTransizioneAvvia`
F5	**Functional**	`GestioneStatoControllerTest.`

5.6.2 Tracciabilità Use Case → Test

I test funzionali e con mock sono tracciati direttamente sui template del Capitolo 2. La Tabella 5.2 sintetizza la copertura dei flussi base e alternativi.

Use Case / Flusso	Test ID
UC1 - Flusso base (prenotazione)	M1
UC1 - Alternativa 3a (slot occupato)	M2, F1
UC1 - Alternativa 4a (capienza)	M3
UC2 - Flusso base (iscrizione)	M6, F2
UC2 - Alternativa 5a (duplicato)	M7, F3
UC3 - Flusso base (avvio sessione)	M4, F4
UC3 - Alternativa 4a (transizione non valida)	M5, F5
UC4 - Flusso base (cambio tariffa)	M8, F6

Tabella 5.2: Matrice di tracciabilità Use Case → Test

5.7 Risultati

La suite di test è composta da **12 classi di test** e **37 test methods**, distribuiti su 4 livelli (unit, mock, integrazione, funzionale). La code coverage è misurabile tramite il report JaCoCo generato automaticamente in fase di build (`target/site/jacoco/`).

5.8 Conclusioni

L'approccio metodologico adottato, guidato dai principi dell'Object-Oriented Design e supportato dall'uso estensivo dei Design Pattern, ha permesso di sviluppare **EscapeManager** come un sistema robusto, altamente coeso e disaccoppiato. L'introduzione di **Mockito** ha permesso di testare i Controller in totale isolamento dal database, garantendo test rapidi e deterministici. La suite di collaudo ha confermato la solidità della logica di business e la corretta integrazione con il layer relazionale, soddisfacendo integralmente i requisiti posti in fase di analisi.

6. Qualità del Software

In questo capitolo si analizza la qualità del software prodotto secondo le metriche e i principi più rilevanti in ambito di Ingegneria del Software, con riferimento allo standard ISO/IEC 25010 (SQuaRE) e ai principi SOLID dell'Object-Oriented Design.

6.1 Aderenza ai Principi SOLID

L'architettura di *EscapeManager* è stata progettata seguendo sistematicamente i principi SOLID:

- **Single Responsibility Principle (SRP):** ogni classe ha una responsabilità ben definita. I Controller coordinano i casi d'uso, i DAO gestiscono la persistenza, il Domain Model incapsula la logica di business e la CLI gestisce l'interazione utente.
- **Open-Closed Principle (OCP):** nuovi stati (`RoomState`) o nuove strategie di prezzo (`PricingStrategy`) possono essere aggiunti creando nuove classi concrete senza modificare il codice esistente della `Stanza` o dei Controller.
- **Liskov Substitution Principle (LSP):** le sottoclassi di `Utente` (Cliente, GameMaster, Admin) possono essere sostituite nelle interfacce che accettano il tipo base senza alterare il comportamento atteso del sistema.
- **Interface Segregation Principle (ISP):** le interfacce DAO sono specifiche e minimali. `WaitingListDAO` espone solo 2 metodi, `StanzaDAO` e `PrenotazioneDAO` estendono `GenericDAO` con metodi mirati.
- **Dependency Inversion Principle (DIP):** tutti i Controller dipendono da *interfacce* DAO iniettate tramite costruttore, non da implementazioni concrete. Questo è il fondamento della testabilità con Mockito.

6.2 Analisi ISO/IEC 25010

Lo standard ISO/IEC 25010 definisce otto caratteristiche di qualità del software. La tabella 6.1 sintetizza la valutazione del sistema rispetto alle caratteristiche più rilevanti per il dominio.

Caratteristica	Descrizione e realizzazione nel progetto	Livello
Adeguatezza Funzionale	Tutti e 4 gli UC (UC1–UC4) sono implementati e testati con test funzionali tracciati.	Alto
Affidabilità	Le eccezioni sono gestite con gerarchia custom e validazioni a ogni livello. Le operazioni DAO usano try-with-resources per prevenire resource leak.	Alto
Usabilità	La CLI guida l'utente con menu espliciti, validazione input e messaggi di errore chiari. I mockup JavaFX mostrano l'interfaccia progettata.	Medio
Efficienza	Query mirate con PreparedStatement. Dataset contenuto. Scalabile con indici aggiuntivi (es. su `data_ora`).	Medio
Manutenibilità	Architettura a strati, pattern GoF, interfacce DAO, DI tramite costruttore. Aggiungere una nuova feature richiede impatto minimo.	Alto
Portabilità	Java + JDBC + configurazione esterna (`db.properties` o env vars). Eseguibile su Windows, macOS, Linux.	Alto

Tabella 6.1: Valutazione ISO/IEC 25010 del sistema EscapeManager

6.3 Code Coverage (JaCoCo)

Per quantificare la copertura del codice, è stato integrato il plugin **JaCoCo** (Java Code Coverage) nel ciclo di build Maven. Il report viene generato automaticamente ad ogni esecuzione della suite di test tramite il comando:

```
mvnw.cmd test
```

Il report HTML risultante è consultabile in `target/site/jacoco/index.html`.

La suite di test comprende **37 test cases** distribuiti su 4 livelli:

- **Unit Test (White-Box):** 22 test sul Domain Model (State, Strategy, Observer, Builder).
- **Mock Test (Mockito):** 15 test sui Controller in isolamento dal DB.
- **Integration Test (DAO):** test di persistenza sul database reale.
- **Functional Test (Black-Box):** test end-to-end tracciati sugli UC templates.

6.4 Affidabilità e Coerenza dei Dati

La coerenza del sistema è garantita da:

- **Validazioni nel layer di controllo:** ogni Controller verifica pre-condizioni e regole di business prima di operare sul DB.
- **Vincoli nel database:** chiavi primarie, chiavi esterne con `ON DELETE CASCADE`, constraint `NOT NULL` e `UNIQUE` su campi critici (es. email).
- **Eccezioni applicative:** la gerarchia `EscapeManagerException` propaga errori in modo controllato, evitando stati intermedi non consistenti.

- **State Pattern:** le transizioni illegali sono bloccate a livello di dominio (`IllegalStateException`), impedendo operazioni incoerenti.

6.5 Manutenibilità ed Estendibilità

La combinazione di architettura a strati, DAO Factory, Dependency Injection e pattern comportamentali consente di introdurre nuove funzionalità con impatto minimo. Esempi concreti:

- Una nuova strategia tariffaria (es. `TariffaNotturna`) richiede solo una nuova classe che implementa `PricingStrategy` e un mapping nel DAO — nessuna modifica ai Controller.
- Un nuovo stato operativo (es. `InPreparazioneState`) richiede solo una nuova classe che implementa `RoomState` — il codice di `Stanza` resta invariato (OCP).
- La migrazione a un diverso DBMS richiede solo una nuova `DAOFactory` concreta.

6.6 Sicurezza e Privacy (Baseline)

Il modello dati prevede il campo `password_hash` per le credenziali utente. La gestione completa dell'autenticazione e autorizzazione non fa parte del prototipo e viene trattata come estensione futura (Capitolo 7).

6.7 Limiti del Prototipo

Il sistema è un proof-of-concept focalizzato sui casi d'uso principali. Alcuni aspetti sono volutamente semplificati: autenticazione reale, gestione avanzata degli slot, notifiche automatiche della lista d'attesa e amministrazione completa delle sedi.

7. Conclusioni e Sviluppi Futuri

7.1 Obiettivi raggiunti

Il progetto *EscapeManager* ha raggiunto gli obiettivi prefissati nella fase di analisi:

- **Architettura solida:** l'applicazione è strutturata secondo il paradigma BCE (Boundary-Control-Entity) con una rigida separazione delle responsabilità tra i livelli di presentazione, controllo, dominio e persistenza.
- **Design Pattern GoF:** sono stati applicati e documentati 5 pattern fondamentali: **State** (ciclo di vita della Stanza), **Strategy** (politiche di prezzo), **Observer** (lista d'attesa), **Builder** (costruzione fluente delle Prenotazioni) e **Singleton** (gestione centralizzata della connessione DB).
- **Pattern architetturali:** DAO, Abstract Factory (DAO Factory) e Dependency Injection sono utilizzati per disaccoppiare la logica di business dalla tecnologia di persistenza.
- **Persistenza relazionale:** il database PostgreSQL, progettato con Single Table Inheritance e junction table, supporta integralmente il Domain Model.
- **Testing multi-livello:** la suite di collaudo comprende test unitari (White-Box), test con mock (Mockito), test di integrazione (DAO) e test funzionali (Black-Box) tracciati sui casi d'uso, per un totale di 37 test cases.

7.2 Sviluppi Futuri

L'architettura disaccoppiata consente di estendere il sistema in diverse direzioni:

1. **Interfaccia Grafica Completa (JavaFX):** i mockup attuali possono essere trasformati in una GUI funzionale collegata ai Controller esistenti, senza modificare il Domain Model.
2. **Autenticazione e Autorizzazione:** il campo `password_hash` è già predisposto nel modello dati. L'implementazione di un sistema di login con hashing sicuro (es. BCrypt) e controllo degli accessi basato sul ruolo (`ruolo` in tabella `UTENTE`) è il passo successivo naturale.
3. **Notifiche automatiche (Observer):** l'infrastruttura Observer per la lista d'attesa è implementata a livello di dominio. L'estensione prevede l'invio automatico di notifiche (email/push) quando uno slot viene liberato (cancellazione prenotazione).
4. **API REST:** l'esposizione di endpoint RESTful tramite un framework leggero (es. Javalin o Spring Boot) permetterebbe l'integrazione con frontend web o mobile, mantenendo intatta la Business Logic.
5. **Adapter Pattern per IoT:** l'integrazione di sensori fisici nelle stanze (es. timer di sessione, rilevatori di completamento enigma) potrebbe essere realizzata tramite il pattern *Adapter*, adattando l'interfaccia dei dispositivi al Domain Model esistente.

6. **Gestione multi-sede avanzata:** l'entità `Sede` è già presente nel database. Un modulo di amministrazione centralizzata per la gestione di più filiali rappresenta un'estensione modulare del sistema.